DOCUMENTATION · RUN IT

Installation and setup

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=INSTALLATION_AND_SETUP. This copy was made on 4 November 2026.

Requirements

COMPONENT	VERSION	PURPOSE
Python	3.11+	Runtime
Java	11+ (only for H2)	Runs the H2 JDBC driver via JPype

Install

```shell
pip install queryapigate                    # SQLite only
pip install "queryapigate[postgres,mysql]"  # add the drivers you need
pip install "queryapigate[all]"             # every driver
```

Available extras: `mysql`, `postgres`, `clickhouse`, `h2`, `all`, `server` (gunicorn) and `dev`. Each database driver is imported only when a connection of that type is used, so you never need drivers you do not use.

DATABASE	DRIVER	EXTRA
MySQL	`mysql-connector-python`	`mysql`
PostgreSQL	`psycopg2-binary`	`postgres`
ClickHouse	`clickhouse-driver`	`clickhouse`
SQLite	`sqlite3` (standard library)	-
H2	`JayDeBeApi` + `JPype1`	`h2`

From source

```shell
git clone https://github.com/AnanthaRajuC/QueryAPIGate.git && cd QueryAPIGate
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
```

Set up a data folder

QueryAPIGate keeps its state in one folder - `QUERYAPIGATE_HOME`, by default the current directory:

```text
queryapigate.db      connections, saved queries, API keys, roles and the audit log (SQLite)
```

```shell
mkdir my-api && cd my-api
queryapigate init            # writes queryapigate.db (a template connection per db type, all inactive)
# edit them: admin UI, or PATCH /api/v1/connections/<name> - set "active": true on the ones you want
queryapigate serve           # http://127.0.0.1:5000
```

To try it without any database of your own, use the bundled examples instead: `queryapigate init && queryapigate examples load && queryapigate serve`.

Configuration

Behaviour is controlled by environment variables - see the table in CONFIGURATION.md (`QUERYAPIGATE_HOME`, `QUERYAPIGATE_ALLOW_WRITES`, `QUERYAPIGATE_API_KEY`, `QUERYAPIGATE_MAX_PAGE_SIZE`, `QUERYAPIGATE_QUERY_TIMEOUT`, `QUERYAPIGATE_POOL_SIZE`, `QUERYAPIGATE_POOL_IDLE_TIMEOUT`, `QUERYAPIGATE_CORS_ORIGINS`, `QUERYAPIGATE_RATE_LIMIT`, `QUERYAPIGATE_TRUST_PROXY`, `QUERYAPIGATE_HOST`, `QUERYAPIGATE_PORT`, `QUERYAPIGATE_DEBUG`, `QUERYAPIGATE_H2_JAR`, `QUERYAPIGATE_REDIS_URL`, `QUERYAPIGATE_DATABASE_URL`).

Running in production

`queryapigate serve` uses Flask's development server. For production use gunicorn with **one worker** and several threads, behind a TLS-terminating reverse proxy - one worker because `/metrics` is per process (a scraper would reach one worker at random), and, without Redis, so are rate limits. For more capacity run more instances, each with one worker (Scaling out):

```shell
pip install "queryapigate[server]"
QUERYAPIGATE_API_KEY=change-me gunicorn --bind 127.0.0.1:5000 --workers 1 --worker-class gthread --threads 8 \
  --timeout 120 "queryapigate.app:create_app()"
```

Behind a reverse proxy or load balancer, also set `QUERYAPIGATE_TRUST_PROXY=1` (the number of proxies) so rate limits and redirects use the real client address and scheme. Once every client reaches it over HTTPS, `QUERYAPIGATE_HSTS_MAX_AGE` (seconds) adds HSTS - only to requests that arrived over HTTPS.

Or use the published Docker image (`ghcr.io/anantharajuc/queryapigate`, with a `-h2` variant that includes Java) or the Dockerfile - see the README. The image sets gunicorn's worker timeout to 120 seconds; keep it above `QUERYAPIGATE_QUERY_TIMEOUT` if you run your own gunicorn. For a full production Docker setup - reverse proxy, secrets, backups, monitoring - see DEPLOYMENT.md.

Shared response cache (Redis)

A saved query's `cache_ttl` (see the README) is served from an in-process cache by default - fast, zero setup, but wiped by every restart and never shared if you run more than one instance behind a load balancer. For an OLTP query that gets hit hundreds of times a second, set `QUERYAPIGATE_REDIS_URL` to point at a Redis instance instead: the

cache then survives a restart and is shared across every instance that points at the same Redis, so a cache warmed by one instance serves every instance's traffic instead of each duplicating the same database hits.

```shell
pip install "queryapigate[redis]"
QUERYAPIGATE_REDIS_URL=redis://localhost:6379/0 queryapigate serve
```

If Redis is briefly unreachable, a cached response is simply treated as a miss - the query still runs against the real database, it's just not served from cache for that one request.

The same `QUERYAPIGATE_REDIS_URL` also shares **rate limits**: the server-wide limit per client address and every key's own `rate_limit` are counted in Redis, so several instances (and `queryapigate mcp` / `events`) enforce one budget, not one each. If Redis fails a check, that process counts on its own for a few seconds before trying Redis again - limits stay enforced, per process, rather than lifted or turned into refusals - logs a warning (at most once a minute), counts it in `queryapigate_rate_limit_fallbacks_total`, and raises the `rate_limits_not_shared` alert for ten minutes.

An optional sidecar service for `docker-compose.yml`:

```yaml
services:
  redis:
    image: redis:7-alpine
  queryapigate:
    # ...
    environment:
      QUERYAPIGATE_REDIS_URL: redis://redis:6379/0
    depends_on:
      - redis
```

Shared metadata store (PostgreSQL)

Everything QueryAPIGate itself keeps - connections, saved queries and their run history, API keys, roles and the audit log - lives in `queryapigate.db`, a SQLite file in `QUERYAPIGATE_HOME`. That is the right default: nothing to install or run, and fast for a single server.

Set `QUERYAPIGATE_DATABASE_URL` to keep all of it in a PostgreSQL database instead when you need:

- **more than one instance** - every instance pointed at the same database serves the same connections, saved queries and keys, and a key revoked through one is revoked on all of them at once;
- **the store on a managed database** - backed up, replicated and failed over by your existing PostgreSQL setup rather than by copying a file.

```shell
pip install "queryapigate[postgres]"
export QUERYAPIGATE_DATABASE_URL=postgresql://queryapigate:secret@db.internal:5432/queryapigate
queryapigate migrate-to-postgres   # once: copies this home's queryapigate.db across (optional)
queryapigate serve
```

- The tables are created automatically on first start. Use a database (or schema) of its own: the account needs to create tables there. To use a schema other than `public`, add `?options=-csearch_path%3Dmyschema` to the URL.

- `migrate-to-postgres` copies everything in one transaction and only ever fills an **empty** database - it refuses, changing nothing, if the target already holds data. `queryapigate.db` is opened read-only and left as it was; `--from PATH` copies a different file.
- Each process keeps one connection per worker thread, so plan PostgreSQL's `max_connections` for `instances × workers × threads` (the Docker image: 1 worker × 8 threads per instance).
- Run history writes skip the store-wide lock and don't wait for PostgreSQL's WAL flush (`synchronous_commit` off for those writes only) - the same trade-off SQLite's default here makes: a crash of the database server can lose the last few runs' history entries, never corrupt anything or lose a configuration change.
- To keep every run for a while rather than each version's newest 50 - for tracking down an API problem after the fact - set `QUERYAPIGATE_HISTORY_RETENTION_DAYS` (e.g. `30`) and browse it with `GET /api/v1/history`. On a very busy server, `QUERYAPIGATE_HISTORY_SAMPLE_RATE` keeps a fraction of successful runs while still recording every failure.
- Back it up with `pg_dump` and restore with `pg_restore` - see Backups and restores, which also lists the secrets a backup doesn't contain. On a SQLite store, `queryapigate backup FILE` takes a consistent copy while the server runs.
- Legacy pre-SQLite files (`db_connections.json`, `saved_sql/`, `api_keys.json`, …) are never read by a PostgreSQL store - and since 0.15 not by any store: bring them into `queryapigate.db` with 0.14 first.

Rate limits are shared between instances through Redis (`QUERYAPIGATE_REDIS_URL`, below). What is **not** shared: `/metrics`, which is per process, and the main server's own `GET /events`. For live events across instances, run `queryapigate events` - it reads every instance's runs from the shared store, and is woken by a PostgreSQL notification as soon as they are written. Point every instance at the same `QUERYAPIGATE_REDIS_URL` to share the response cache too.

Scheduled exports to a file

`queryapigate export <query> --out <path>` runs a saved query and writes its full result to a file, entirely in-process against `QUERYAPIGATE_HOME` - no server needs to be running, no HTTP round trip, no API key. It's built for cron, a systemd timer or a Kubernetes CronJob to call, not a scheduler itself - scheduling, retries and failure notification stay exactly where they already work well:

```shell
queryapigate export top_rented_films --out '/exports/{name}_{date}.csv'
```

`{name}` (the saved query's name) and `{date}` (`YYYY-MM-DD`) in `--out` are filled in; the target directory is created if missing. `--format` is `csv` (default), `tsv`, `ndjson` or `parquet` - the same formats `?stream=true` supports, since this calls the same streaming code path internally rather than shelling out to `curl` against itself. `--connection` overrides the saved query's own default connection; `--param name=value` (repeatable) supplies a required parameter. The result is written to a temporary file in the same directory and renamed into place only once it's complete, so a failed run never leaves a partial or corrupt file at the final path - and exits non-zero on any failure, so cron's own failure handling (mail, an alerting integration, whatever the operator already has) works unmodified:

```shell
# crontab: every morning at 6am, mail on failure (cron's own default behaviour)
0 6 * * * /usr/local/bin/queryapigate export top_rented_films --out '/exports/{name}_{date}.csv'
```

Verify

```shell
curl http://127.0.0.1:5000/health
curl http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: <your admin key>'
```

Running the tests

```shell
pip install -e ".[dev]"
ruff check .
python -m unittest discover -s tests -t .
```

Integration tests against real databases are enabled by setting `QUERYAPIGATE_IT_POSTGRES`, `QUERYAPIGATE_IT_MYSQL`, `QUERYAPIGATE_IT_CLICKHOUSE` and/or `QUERYAPIGATE_IT_H2` to a JSON connection object - see the header of `tests/test_integration.py`. `tests/test_sql_guard_fuzz.py` fuzzes the SQL guard and parameter binder with Hypothesis and always runs as part of the suite above.